
1. Introducción

Hoy en día, casi todos escribimos en computadoras, celulares y tablets, y los **correctores ortográficos** se han vuelto súper importantes. Nos ayudan a **detectar errores de tipeo y sugerir la palabra correcta**, evitando confusiones y errores tontos (Jurafsky & Martin, 2023).

En este proyecto hice un **corrector simple** que:

1. Corrige errores a **una sola edición** (inserción, eliminación, sustitución o transposición).
2. Ordena las sugerencias usando **frecuencia** del diccionario.
3. Funciona solo con listas y tuplas, sin librerías externas complejas.

La idea era entender **cómo funciona un corrector básico** y aplicarlo en un programa real que podemos ejecutar y probar con diferentes palabras.

2. Marco Teórico y Explicación del Código

2.1 Distancias y operaciones de edición

Un concepto clave en los correctores ortográficos es la **distancia de edición**, que básicamente nos dice cuántos cambios necesitamos hacer para transformar una palabra en otra (Levenshtein, 1966).

Las operaciones principales son:

- **Inserción**: agregar un caracter.
- **Borrado**: eliminar un caracter.
- **Sustitución**: cambiar un caracter por otro.
- **Transposición**: intercambiar dos caracteres adyacentes (propio de Damerau-Levenshtein).

En mi código, la función `generar_variantes_1ed()` implementa estas operaciones para crear **todas las palabras a una edición** de la ingresada. Por ejemplo, si escribimos casa:

- **Eliminación**: asa, csa, caa, cas
- **Inserción**: acasa, bcasa, ... , casaa, casab, ...
- **Sustitución**: basa, masa, casa (exceptuando la misma letra)
- **Transposición**: acsa, csaa, caas

Estas operaciones son las que usan los correctores modernos, pero aquí lo hacemos de forma **manual** con listas y bucles, lo cual me ayudó a entender bien cómo funcionan.

Diferencias entre Levenshtein y Damerau-Levenshtein:

- Levenshtein **no corrige transposiciones**.
- Damerau-Levenshtein sí, y por eso es más útil para errores humanos comunes al teclear rápido.

El corrector incluye transposición, así que se parece más a Damerau-Levenshtein.

2.2 Corrección de una sola edición

Limitarse a **una sola edición** funciona bien en muchos casos, porque la mayoría de los errores que cometemos al escribir son **de un solo carácter**, ya sea por teclear rápido, equivocarnos de tecla o distracción.

Ejemplo con mi código: Si escribimos `camio`, la función `generar_variantes_1ed()` genera variantes y luego `filtrar_contra_dicc()` selecciona solo las que existen en el diccionario.

Resultado: cambio, camino. Esto es suficiente para que el usuario vea la palabra correcta entre las primeras sugerencias.

Limitación: errores múltiples, como `grqande` en lugar de `grande`, no se corrigen con una sola edición. Esto muestra por qué los correctores más avanzados usan múltiples ediciones y modelos de lenguaje.

2.3 Ranking por frecuencia y contexto

Después de generar las variantes, necesitamos decidir **qué sugerencia mostrar primero**. Usamos la **frecuencia del diccionario** para esto, con la función `ordenar_por_frecuencia()`.

Ejemplo:

Palabra ingresada: hte

- Variantes generadas: the, hat, hit
- Diccionario con frecuencias: the(5000), hat(1500), hit(1000)
- Ranking final: the, hat, hit

Si solo usamos frecuencia, **el contexto se pierde**: por ejemplo, la palabra correcta podría no tener sentido en la frase. Incluir contexto haría el corrector más preciso, pero también mucho más complejo.

2.4 Normalización de palabras

Antes de corregir, necesitamos que las palabras estén en un formato consistente. La función `normalize(word)` hace:

- Convierte todo a **minúsculas**.
- Reemplaza letras con acentos (á, é, í, ó, ú) por versiones sin acento.
- Mantiene la letra ñ.

Ejemplo:

```
normalize("Çassa") → "cassa"  
normalize("niño") → "niño"  
normalize("café") → "cafe"
```

Esto asegura que las palabras ingresadas coincidan con el diccionario y evita perder sugerencias por acentos o mayúsculas.

2.5 Evaluación de correctores

Para medir qué tan bien funciona el corrector usamos **métricas Top-k**:

- **Accuracy@k**: cuántas veces la palabra correcta aparece entre las primeras k sugerencias.
- **Recall@k**: probabilidad de que la palabra correcta esté en top-k.
- **MRR (Mean Reciprocal Rank)**: promedio de la inversa de la posición de la primera sugerencia correcta.

Ejemplo:

Palabra: camio

- Top-3 sugerencias: cambio, camino, camio
- Palabra correcta: cambio
- $\text{Accuracy}@1 = 1$, $\text{Recall}@3 = 1$, $\text{MRR} = 1/1 = 1$

Estas métricas ayudan a saber si el corrector es útil y rápido para el usuario.

2.6 Aplicaciones y limitaciones

Aplicaciones:

- Teclados de celulares (autocorrección).
- Buscadores web (sugerencias mientras escribes).
- Formularios online (evita errores tipográficos).

Limitaciones:

- Palabras poco frecuentes pueden no aparecer en top-k.
 - Nombres propios pueden corregirse mal (Isac → Isaac).
 - Jerga o abreviaturas no están en el diccionario.
-

2.7 Leer el Diccionario

La función `leer_diccionario(path)` carga `word_freq.txt` con palabras y frecuencia ('palabra': frecuencia,).

- Normaliza cada palabra.
- Convierte frecuencia a entero.
- Ordena por palabra y combina duplicados tomando la frecuencia mayor.

Ejemplo:

```
'casa': 5000,  
'gato': 3000,  
'perro': 2500,
```

Esto da un diccionario limpio y eficiente.

2.8 Búsqueda Binaria

Para buscar palabras rápido usamos `binaria_tuplas(dicc, objetivo)`.

- Devuelve el índice si la palabra existe.
- Devuelve -1 si no existe.

Ejemplo:

```
dicc = [('casa', 5000), ('gato', 3000), ('perro', 2500)]
binaria_tuplas(dicc, 'gato') → 1
binaria_tuplas(dicc, 'vaca') → -1
```

Esto hace que la búsqueda sea rápida incluso con diccionarios grandes.

2.9 Generar Variantes a Una Edición

`generar_variantes_1ed(w, alfabeto)` crea todas las palabras que están a **una edición** de la ingresada.

Operaciones:

1. **Eliminación:** cada carácter se elimina.
2. **Inserción:** inserta letras del alfabeto en cada posición.
3. **Sustitución:** cambia cada letra por otra.
4. **Transposición:** intercambia letras adyacentes.

Ejemplo:

Palabra: casa

- Eliminación: asa, csa, caa, cas
 - Inserción: acasa, bcasa, ...
 - Sustitución: basa, masa, ...
 - Transposición: acsa, csaa, caas
-

2.10 Filtrar contra el Diccionario

`filtrar_contra_dicc(variantes, dicc)` mantiene solo las palabras que **existen en el diccionario** usando búsqueda binaria.

Ejemplo:

Palabra ingresada: casaa

Variantes generadas: casa, cassa, csa, ...

Palabras válidas en diccionario: casa → candidato.

2.11 Duplicar y Ordenar

`deduplicar_ordenado(cands)` elimina duplicados y toma la **frecuencia mayor**.

`ordenar_por_frecuencia(cands)` organiza candidatos por **frecuencia descendente**.

Ejemplo completo:

Palabra: camio

- Variantes: cambio, camino, camio
 - Diccionario: cambio(5000), camino(3000)
 - Ranking final: cambio, camino
-

2.12 Ejecución del Programa

El programa principal (main) hace:

1. Toma palabra y k (Top-k).
2. Normaliza la palabra.
3. Busca si ya existe en el diccionario.
4. Si no existe, genera variantes a 1 edición.
5. Filtra y ordena candidatos.
6. Muestra Top-k sugerencias.

Ejemplo:

```
python main.py hte 3
```

Salida:

```
Sugerencias Top-3 para 'hte':  
1. the (freq=79809)  
2. he (freq=12401)  
3. ate (freq=21)
```

3. Resultados Esperados y Limitaciones

Resultados:

- Correcciones rápidas para errores de un carácter.
- Ranking fácil de entender.
- Funciona solo con listas y tuplas, sin librerías externas.

Limitaciones:

- Solo acepta palabras en inglés
 - No corrige errores de más de una edición.
 - Puede fallar con palabras poco frecuentes, nombres propios o jerga.
 - Los acentos pueden perder info si no se normalizan bien.
-

4. Conclusión

Este proyecto muestra cómo construir un **corrector ortográfico simple**, usando **una edición y ranking por frecuencia**, solo con **listas y tuplas**. Enseña:

- Distancias de edición y operaciones básicas.
- Normalización de texto y manejo de acentos.
- Cómo filtrar, deduplicar y rankear sugerencias.

Aunque simple, refleja **limitaciones reales**, sesgos y la influencia de la frecuencia en las correcciones.

6. Referencias (APA)

Damerau, F. J. (1964). *A technique for computer detection and correction of spelling errors*. Communications of the ACM, 7(3), 171–176.

Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing* (4th ed.). Pearson.

Levenshtein, V. I. (1966). *Binary codes capable of correcting deletions, insertions, and reversals*. Soviet Physics Doklady, 10(8), 707–710.

Norvig, P. (2007). *How to write a spelling corrector*. <http://norvig.com/spell-correct.html>